Secure Login Portal

REVISED TECHNICAL & IMPLEMENTATION PROPOSAL

FastAPI • Azure App Service • Managed Identity • Azure AI Foundry

RFP reference: MMAUG-AIDB-RFP-002 | Status: Proposed solution for owner approval

Prepared for: MMAUG AI & DevOps Bootcamp project owners | Delivery model: Solo developer

Delivery target: Four weeks (20 working days) | Version: 3.0 | 08 November 2026

Executive decision summary

This revision responds to the original MMAUG High School participant RFP and incorporates subsequent project-owner feedback. It proposes a secure, role-based school portal using React/TypeScript, Python FastAPI, PostgreSQL, Azure App Service and three role-specific Azure AI Foundry agents. Azure Front Door has been removed. Azure Application Gateway with Web Application Firewall (WAF) is an OPTIONAL regional protection layer, subject to cost and networking decisions.

A user-assigned managed identity (UAMI) will be manually created as a bootstrap step. GitHub Actions will federate to the approved deployment identity through GitHub OpenID Connect (OIDC) and Microsoft Entra ID to obtain short-lived Azure tokens: no client secrets, passwords or publish profiles will be stored for deployment. Workload identity and Azure RBAC remain distinct; a UAMI does not itself grant resource-creation permissions.

The four-week commitment is a synthetic-data, production-oriented MVP with verifiable security and DevOps controls. A production rollout handling real minors' data requires a separate readiness decision.

1. Requirement baseline and approved-change register

Source	Requirement / decision	Treatment in this proposal
Original RFP	Student, teacher and administrator roles; sign-in/out; recovery; dashboards; protected API; account administration; audit; CI/CD	Mandatory MVP acceptance scope
Original RFP	Five students, two teachers, one administrator; invalid-access demonstrations; 1,500-account scale discussion	Explicit final demo test cases
Owner feedback	Contributor role on subscription; manually create managed identity; use managed identity for project	Manual UAMI bootstrap; workload identity; privileged RBAC grants require additional authorization
Owner feedback	Foundry agents for Student, Teacher, Administrator	Three agent experiences with backend-mediated access and role isolation
Owner feedback	Use an Azure App Service Plan	Frontend and FastAPI deployed as two App Services sharing a Linux plan, subject to SKU/quota
Owner feedback	No Azure Front Door; optionally Application Gateway + WAF	Front Door removed; Application Gateway/WAF drawn and budgeted as optional
Latest owner feedback	Federate the manually created identity to GitHub using OIDC; deployment must use no secrets	Federated identity credential, GitHub environments, azure/login with OIDC, Terraform access via short-lived tokens

2. Business outcome and scoped MVP

The simulated school has approximately 1,200 students and 150 staff. The portal will provide centralized access, least-privilege controls and visibility into sign-in and administration events. It is not a full student information system.

Area	Minimum functional demonstration
Identity	Managed sign-in/sign-out, failed sign-in behavior, tested account recovery and administrator MFA
Authorization	Student/teacher/admin dashboard separation; protected FastAPI routes; object-level restrictions; 401/403 cases
Administration	View accounts; activate/deactivate application access; permitted role changes; audit each change
Data and audit	Synthetic user profiles, roles and events in PostgreSQL; no stored plaintext passwords
Agents	Student, Teacher and Administrator agent access via FastAPI; role-appropriate instructions, tools and data boundaries
DevSecOps	GitHub source; automated builds, tests, security validation, deployment verification and rollback
Operational proof	Monitoring and alerts; dev/UAT/prod separation plan; 1,500 registered-account scalability discussion

Excluded from the four-week MVP: real student data, grades/attendance/fees, school-wide launch, third-party SIS integration, autonomous high-risk administrative actions and guaranteed high availability without separate engineering validation.

3. Reference technology choices

Layer	Service / technology	Design intent
Frontend	React + TypeScript on Azure App Service	Role-aware web UI
Backend and REST APIs	Python FastAPI + Pydantic	API-first modular monolith; central authorization
Database	Azure Database for PostgreSQL; SQLAlchemy + Alembic	User profile, role membership and audit storage
Workforce identity	Microsoft Entra ID (assumed; confirm school tenant)	Student/teacher/admin SSO; MFA and recovery policy
Agent runtime	Azure AI Foundry project and three agents	Student/Teacher/Admin role-specific agent experiences
Runtime identity	User-assigned managed identity	App-to-Azure service access after explicit RBAC grants
Hosting and edge	Linux App Service Plan; optional Application Gateway WAF_v2	Regional web/API hosting; optional reverse proxy/WAF
DevOps and observability	GitHub Actions OIDC; Terraform; Key Vault; App Insights / Azure Monitor	Secretless Azure deployment, versioned releases, telemetry and alarms

4. Target architecture — Microsoft Azure

Owner-feedback revision: Azure Front Door removed; Azure Application Gateway + WAF is OPTIONAL. Arrows show conceptual traffic and workload access, not Azure RBAC grants.

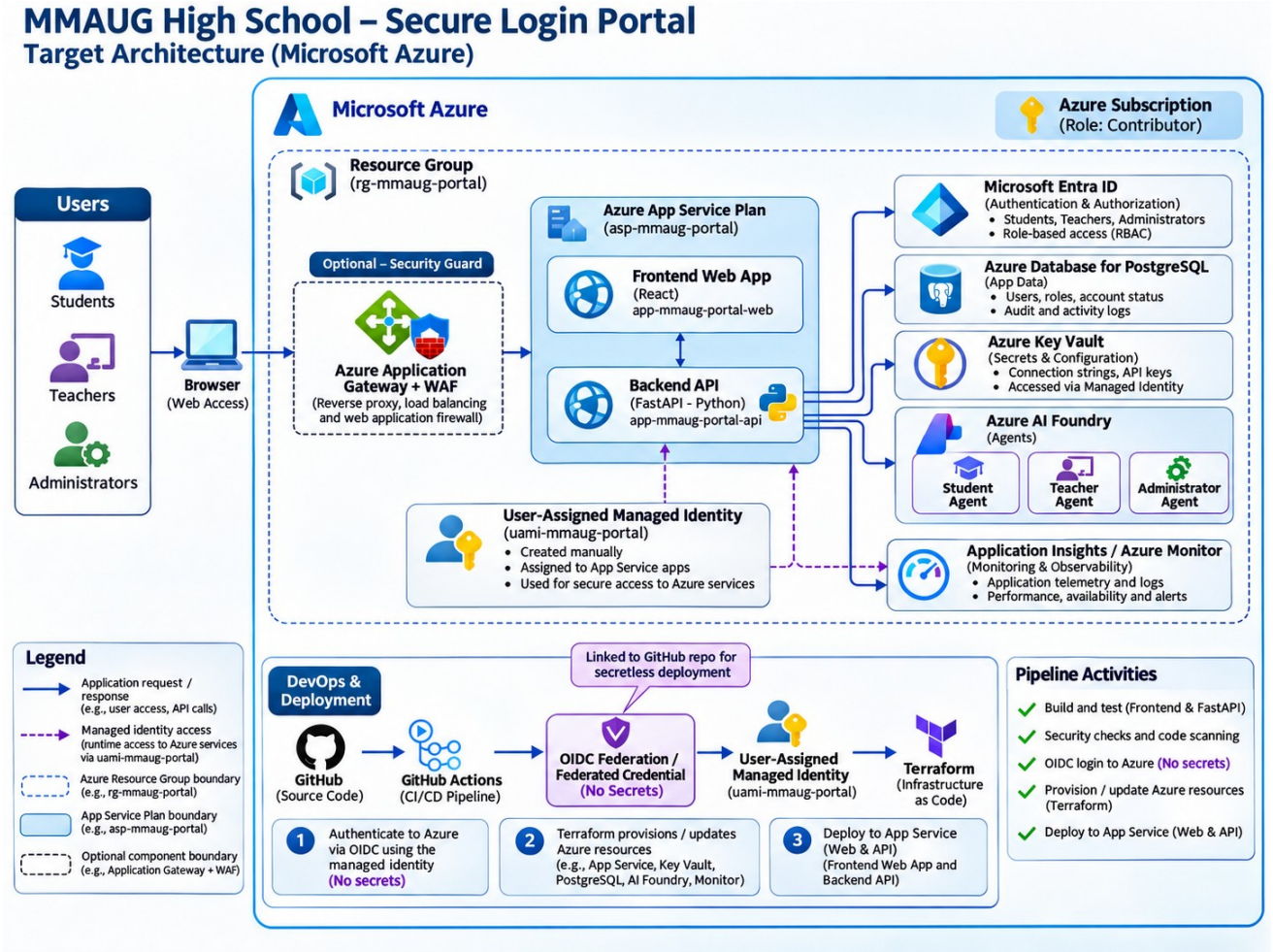


Figure 1. Target architecture — optional Application Gateway/WAF; App Service Plan, React/FastAPI, Entra sign-in, PostgreSQL, three Foundry agents and GitHub OIDC federation to Azure deployment identity. Diagram is conceptual; resource and identity boundaries are clarified in text.

Architecture review note: Microsoft Entra ID is a tenant-level service, not physically contained in the resource group. If optional Gateway/WAF is deployed, prevent direct App Service origin bypass. Secure private database connectivity. Figure 1 is conceptual rather than a full network topology.

5. Authentication, authorization and agent isolation

- Browser signs in using Microsoft Entra ID through OpenID Connect authorization code with PKCE; verify the existing school identity provider before implementation.
- React calls FastAPI with a token issued for that API. FastAPI checks issuer, signature, audience, expiry and required scopes, then looks up account status and authorized application role.
- All protected endpoints and data queries enforce server-side RBAC and, where relevant, object-level authorization. A role switch or modified URL must not elevate privileges.
- FastAPI selects only the agent authorized for the caller (Student, Teacher or Administrator). Each agent receives least-privilege tool access and approved context; the LLM must never be the ultimate access-control decision maker.
- Agents operate read-only on synthetic data in the MVP. Sensitive changes (e.g. user deactivation or role assignment) remain deterministic, authorized FastAPI functions with audit logging and explicit confirmation.
- Trace agent prompts, tool calls and outcomes with privacy-aware logging. Do not include secrets, access tokens or unnecessary student information in logs or prompts.

6. Managed identity and Azure deployment bootstrap

The request to create “everything using managed identity” must be interpreted carefully: a UAMI authenticates a workload, but creating infrastructure requires Azure deployment permissions and role assignments. The proposed sequence is:

- Project owner grants/validates the developer’s subscription or resource-group Contributor access. Contributor does not ordinarily permit creating RBAC role assignments.
- Developer manually creates a dedicated user-assigned managed identity in the authorized resource group.
- A subscription Owner/User Access Administrator or delegated role-assignment administrator grants only necessary roles to the UAMI, e.g. appropriately scoped Key Vault and Foundry access. Validate PostgreSQL Microsoft Entra authentication setup independently.
- Attach UAMI to the FastAPI App Service; attach it to the frontend App Service only if that app itself needs Azure resource access.
- Register a federated identity credential on the manually created UAMI for GitHub Actions (issuer, exact repository/environment subject, audience). Give it only deployment RBAC permissions approved for its resource scope. Prefer a separately scoped runtime UAMI for App Service: reuse of the same identity for both deployment and runtime requires explicit risk acceptance.
- Configure GitHub environment protection and GitHub Actions OIDC permissions; use azure/login to authenticate without client secrets or publish profiles. Configure Terraform AzureRM provider and remote state to use Microsoft Entra/OIDC authorization as supported, never storage account keys in repository secrets.
- Use Terraform to provision subsequent components after validating App Service Plan quota, Foundry region/model availability, authorized role grants and total cost.

7. GitHub OIDC federation: secretless deployment design

The project owner explicitly requires Azure deployments without client secrets. GitHub Actions will exchange a short-lived GitHub OIDC assertion for an Azure token through Microsoft Entra ID. Trust is limited to the intended repository and GitHub environment. Authentication and deployment authorization are separate: the managed identity needs the relevant Azure RBAC grants.

Configuration element	Required design
Azure bootstrap	Manually create uami-mmaug-github-deploy (proposed

	name); capture client ID, tenant ID and subscription ID. Creation itself uses the signed-in operator's authorized identity.
Federated credential	Issuer: https://token.actions.githubusercontent.com; Subject: repo:ORG/mmaug-school-portal:environment:development (replace with actual owner and repo); Audience: api://AzureADTokenExchange.
GitHub environment	Create development; restrict allowed branches and set appropriate approvals. Separate test and production federation subjects and environment policies.
Workflow permissions	id-token: write and contents: read; authenticate using azure/login with client-id, tenant-id and subscription-id as GitHub variables, never AZURE_CLIENT_SECRET.
Terraform	Use OIDC-authenticated AzureRM provider and a secure, access-controlled remote state backend; grant state read/write permissions separately. The state file can contain sensitive attributes.
Deployment authorization	Scope Contributor only where necessary; role assignments, Key Vault RBAC and other privileged grants require authorized owners/delegated admins. Limit production scope.
Runtime access	Attach runtime managed identity to App Services; use it for Key Vault/Foundry and database Entra access where configured. Prefer distinct runtime and deploy identities.

Example acceptance evidence: a GitHub Actions run that signs in using OIDC and performs an authorized Azure read; a Terraform plan/apply using federated authentication; an App Service deployment and health check; and proof that the repository and workflow have no deployment client secret, password or publish profile.

8. Security, privacy and operations

Control	Planned implementation / evidence
Network and ingress	HTTPS; restrict backend and database exposure. Optional Application Gateway WAF_v2 only after network, cost and origin-access design are approved.
Secrets and identity	Key Vault, managed identities and explicit narrowly scoped RBAC. No application secrets in source or build logs.
Data and privacy	Synthetic data only for demonstration. Before live use: privacy review, retention schedule, student-data classification and required DPIA assessment.
Availability and backup	Health probes, diagnostic alerts, PostgreSQL backup/restore drill and rollback of both application and compatible schema.
Testing	Pytest API/authorization tests; UI automation; agent prompt-injection and cross-role data-leak tests; dependency/secret scans.
Release control	GitHub PR review, versioned artifacts, development/UAT/prod environment protections and deployment evidence.

9. Four-week solo execution and acceptance gates

Week	Execution focus	Review evidence
1 — Design and foundation	Requirements, identity questions, architecture, manual deployment UAMI, OIDC federation, repo, frontend/API skeleton	Architecture approval; OIDC sign-in evidence; RBAC blocker list
2 — Working portal	Entra integration, eight synthetic accounts, PostgreSQL, dashboards,	Login and recovery demonstrations; role denial; account audit proof

	protected API, admin status/roles, recovery	
3 — Cloud and agents	App Service Plan, secretless GitHub OIDC CI/CD, Foundry integration, three agent routes, telemetry, security tests	OIDC-driven deployment; three agent demos; isolation and prompt-injection checks
4 — Validation and handover	E2E tests, availability/rollback drill, 1,500-account discussion, documented limits, presentation	Acceptance test record; GitHub release/deployment evidence; complete handover

Delivery risk: Three Foundry agents, identity integration and production-oriented DevSecOps in four weeks are aggressive for one beginner. Prioritize RFP acceptance first and deliver agents as deliberately constrained experiences. Record any unmet requirements rather than claiming production readiness.

10. Owner decisions and dependencies

- Confirm Entra tenant/account source and authorized recovery/MFA journey.
- Confirm deployment UAMI naming, subscription scope, GitHub repository and environments, and who grants Azure RBAC roles (Contributor alone cannot ordinarily grant them).
- Confirm region, App Service Plan SKU/quota and budget ceiling; confirm PostgreSQL/Foundry service availability.
- Approve the exact purpose and permitted data/tools for each Foundry agent.
- Decide whether optional Application Gateway + WAF is in MVP scope; otherwise access App Service securely without it.
- Approve the scope boundary: synthetic-data demonstration now, separate production readiness gate later.
- Confirm whether a second, least-privilege runtime UAMI is authorized; if the single deployment UAMI must also run App Services, document and approve that risk.

11. Final demonstration and handover

Demonstrate five students, two teachers and one admin; sign-in failure and recovery; each dashboard; blocked cross-role API access; permitted admin changes; audit trail; three role-appropriate Foundry agents; an agent denied access to prohibited data; and a GitHub Actions OIDC deployment without stored Azure secrets. Explain planned capacity for 1,500 registered accounts and show test and rollback evidence.

Hand over: requirements traceability, architecture and API specs, threat model/permissions matrix, source and Terraform, OIDC federation settings, RBAC and identity inventory, deployment runbook, CI/CD runs, test results, cost baseline, risk register and known limitations.

Approval requested: endorse revised stack, owner-feedback change register, OIDC/managed-identity/RBAC bootstrap, optional Gateway decision, agent behavior, budget and four-week MVP acceptance criteria.